

SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

LLM agent skills continuously evolve by aggregating cross-user interaction trajectories and applying agentic, evidence-driven skill updates.

Source: arxiv.org/abs/2604.08377

Agent Skills Remain Static After Deployment

Users rediscover solutions independently instead of improving a shared skill ecosystem

Static skill paradigm

Manual installation from a centralized hub

Runtime discoveries rarely persist beyond individual sessions

Similar failures and recoveries recur across users over time

Evolving skill paradigm

Interaction trajectories become reusable evidence

Observed recoveries can update the shared skill library

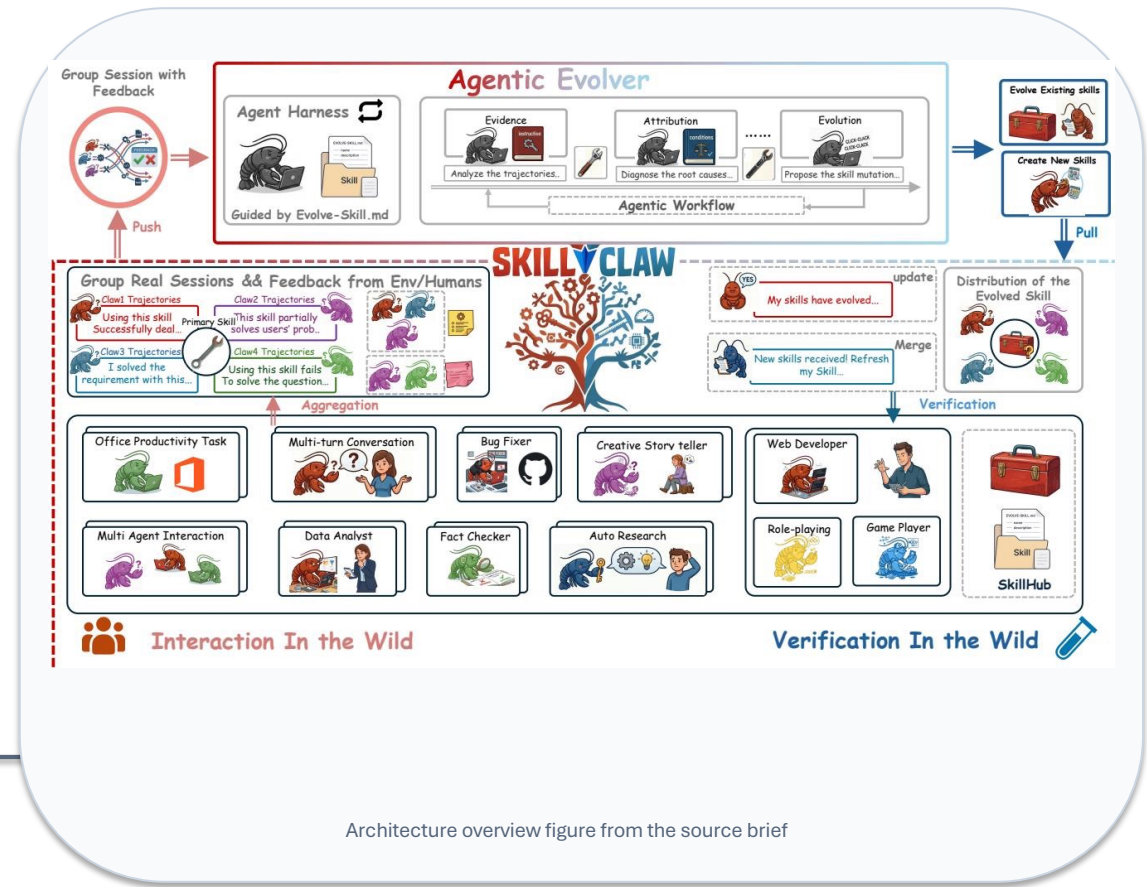
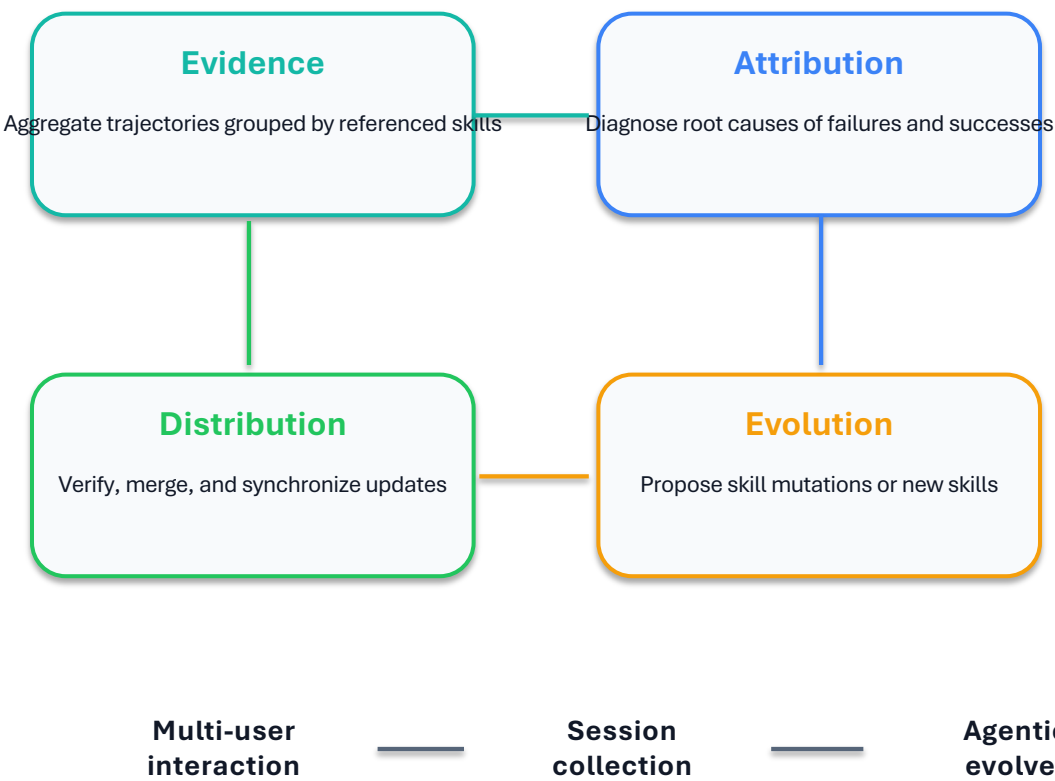
Knowledge accumulates at the system level

Gap in existing approaches

- Memory-based methods store trajectories but remain instance-tied and hard to generalize.
- Skill-based methods compress experience into instructions but treat the library as static.

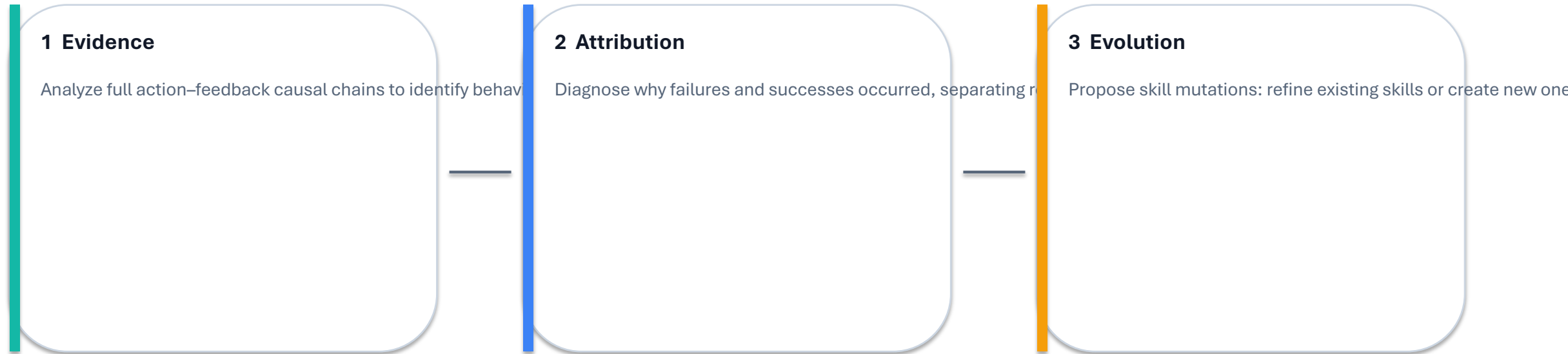
Closed-Loop Pipeline: Multi-User Interaction → Shared Skill Evolution

Centralized evolution architecture with verified synchronization back to all agents



Agentic Evolver: Evidence → Attribution → Evolution

Open-ended agent reasoning over interaction evidence, not predefined update rules

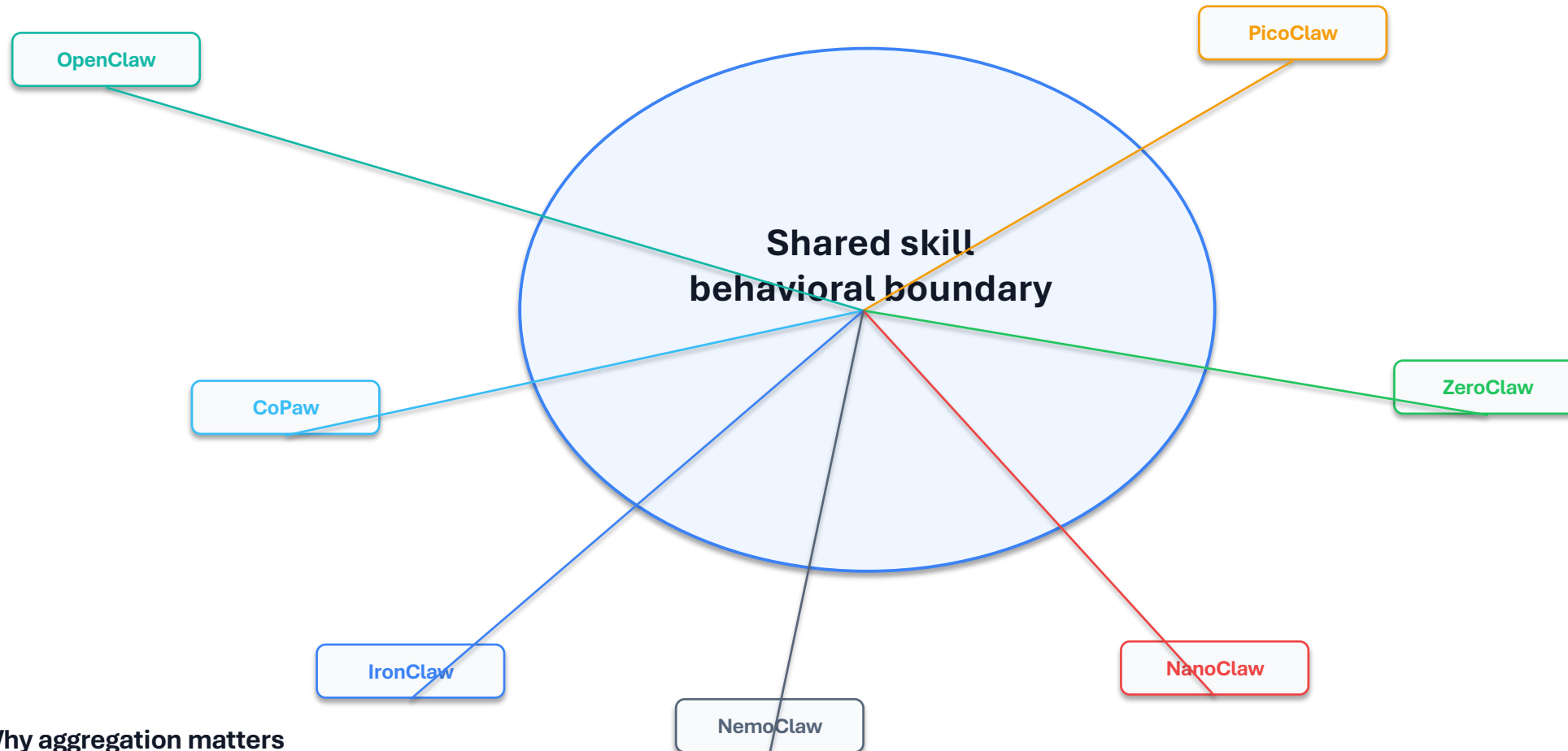


Verification gate

- Candidate skills are verified before they enter the shared repository.
- Rejected candidates keep the current best pool, preventing regressions.
- Accepted updates synchronize back to agents, closing the collective loop.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

Compatible Claw-style systems contribute complementary signals around the same referenced skill



Why aggregation matters

- Diverse contexts reveal both success patterns and failure modes.
- A single user rarely provides enough signal to distinguish generalizable improvements from idiosyncratic fixes.

Case Study: Multimodal Creative Pipeline Skill — 6 Days

Conservative verification accepted only 1 of 6 candidate skills

Accepted: 1 / 6

Rejected: 5 / 6

Date / cycle	Candidate skill	Outcome	Rationale
Day 1	validate-tmp-workspace-inputs	Accepted	Checked /tmp_workspace inputs and environment setup; upgraded to new best pool.
Day 2	multimodal-input-validation-and-setup	Rejected	Multimodal input validation and output environment initialization; kept current best pool.
Day 3	multimodal-creative-task-pipeline	Rejected	Pipeline for extracting from PDF/video/image; kept current best pool.
Day 4	multimodal-creative-task-pipeline (impr.)	Rejected	Added image classification, visual generation, structured validation; kept current best pool.
Day 5	pipeline (impr.) + validate-required-input-files	Rejected	Added per-file fail-fast validation; kept current best pool.
Day 6	multimodal-creative-task-pipeline (cand.)	Rejected	Extended PDF-to-poster/document-to-visual paths; not admitted to next cycle.

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement

Evidence supported 3 accepted updates over 6 days before plateauing

Accepted updates: 3 / 6 days

Plateau / rejected: later cycles

Date / cycle	Candidate skill	Outcome	Rationale
Day 1	git-push-with-auth-fallback	Accepted	Safe fallback instead of blocking on push failure; added to Safety best pool.
Day 2	git-push-with-auth-fallback	Accepted	Unified patch/bundle filenames and reduced inconsistency; kept updated best pool.
Day 3	git-push-with-auth-fallback + git-clone-to-directory	Accepted	Auth-alternative paths and secrets audit; fixed subshell pitfalls.
Day 4	(none)	No new update	Same-pool retest; validator confirmed current pool as best.
Day 5	git-push-with-auth-fallback	Rejected	Added “push hang treated as auth failure”; no improvement.
Day 6	git-push-with-auth-fallback	Rejected	Added identity config and filename consistency; no improvement; not admitted.

Key Takeaways: What Distinguishes SkillClaw

Implications for continuously improving LLM agent systems

1. Collective evolution

Individual interactions improve a shared, continuously improving pool of skills.

2. Fully automatic

Runtime interaction drives evolution without manual curation.

3. Agentic evolution paradigm

Open-ended reasoning over evidence replaces predefined updates.

Implication

Agent systems can treat runtime usage as a continuously expanding evidence base—where verified skill updates accumulate.

- Cross-user aggregation provides the signal.
- Agentic analysis proposes the mutation.
- Conservative verification protects the shared pool.